COMP9044

重要内容	☐
类型	笔记
标签	Empty
状态	Empty
项目	Empty
领域	Empty
网址	Empty
归档	☐
创建时间	2026/02/19 13:31
属性	Empty

9044 python学习

W2 Filters

shell

1 文件录目录操作

ls 列出目录内容

```
ls
ls -l          # 详细信息（权限、大小、时间）
ls -a          # 显示隐藏文件
ls -lh         # 人类可读大小
ls -lah        # 组合
```

cd 切换目录

```
cd /home          # 进入指定目录
```

```
cd ..            # 返回上级目录
cd ~             # 回到用户主目录
cd -             # 回到上一次目录
```

pwd 查看当前路径

mkdir 创建目录

```
mkdir test
mkdir -p a/b/c   # 递归创建 逐层创立 a文件夹下b b文件夹下c
```

rm 删除文件或目录

```
rm file.txt      # 删除文件
rm -r folder     # 删除目录
rm -rf folder    # 强制递归删除（危险）没有回收站
```

cp 复制

```
cp file1 file2 # 将file1文件中的内容复制粘贴到file2文件中
cp file.txt backup/ # 讲file文件复制粘贴到 backup文件夹下
cp -r dir1 dir2 #
```

mv 移动/重命名

```
mv old.txt new.txt # 原来的old消失 创建一个新的new 重命名
mv file.txt /home/user/ #
```

2 文件内容查查看

cat 查看全文

less 分页查看

- q 退出
- /关键字 搜索

head 查看前十行 tail 查看后十行

```
-n 改变输出行数
# 会把第 81-100 行输出到结果中
head -n 100 | tail -n 20
```

标准输出指令

```
echo Hello # 打印Hello到屏幕
# 输出变量
name=“Tom”
echo $name
# echo的输出重定向到file.txt文件中
echo "Hello World" > file.txt
# echo 查看环境变量
echo $HOME
echo $PATH
```

Filters

过滤器是一个从标准输入读取数据、对数据进行转换、并将结果写到标准输出的程序。（一个字节流变换器 byte stream）

filter的核心思想是：stream processing

filrters重定向

> 改变文件描述符默认连接目标

```
# echo 打印到屏幕
echo hello
# echo 写入到文件中（覆盖文件内容）
# 如果没有文件则自动创建一个文件
echo hello > file.txt
# echo 写入到文件中（追加，追加到文件的最后一行）
echo hello >> file.txt
# cat 默认程序从键盘读取输入 ctrl+D结束
cat
# cat 打开文件 file.txt作为cat的参数
cat file.txt
cat file1.txt file2.txt
# cat 从文件中读（file作为cat的输入源）
cat < file.txt
# ls 打印目录文件列表
# 错误重定向。程序有两种输出，正常输出stdout 和 错误输出stderr
ls not_esist.txt # 会报错
```

```
ls not_exist.txt 2> error.txt # 将错误写入到文件中
command > out.txt 2> err.txt #
```

filters的管道

> 重定向是改变文件来源或者去向。管道是吧一个进程的输出连接到另一个进程的输入
>
> ```
> filter1 | filter2 | filter3
> data → f1 → f2 → f3 → result
> ```

filters的特点：

> - 行位单位：按行处理
> - 编码：UTF-8 或 ASCII

filters的参数

> - 选项以 - 引入
> - 短格式 -v
> - 长格式 --version
> - 段格式通常可以组合 -av
> - 帮助 --help 或 -h 可以列出命令行选项

cat 简单过滤器

> - cat 命令会把输入原样复制到输出中
> - cat 给定文件名，会把文件的内容连接（concatenate）后输出到标准输出（stdout）
>
> ```
> cat -n file.txt #  行号（从1开始） + 内容
> cat -A file.txt # 打印出不能打印的字符
> cat -s 合并多个空格行
> ```

正则表达式

> 在ubtun虚拟机种 \home\COMP9044\regulation.txt 中做练习

grep的参数

-E 用扩展正则（推荐用 不用也行）

-n 显示行号

-o 只输出匹配到的内容。不带这个选项会输出带有匹配内容的整行

-v 取反

1. Regular Expressions Basics

```
# 1. 普通字符
grep -E 'login' regulation.txt

# 2. 星号 *前面内容重复0次或多次
echo 'abbbcd' | grep -E 'ab*cd'   # 匹配 a + 0或多 b + cd
echo 'acd' | grep -E 'ab*cd'   # b重复0次 可以匹配到
grep -E 'user=bob *id=' regulation.txt # 不用打空格

# 3并集 匹配三个单词其中之一
grep -E 'INFO|WARN|ERROR' regulation.txt
grep -E ' (INFO|WARN|ERROR) ' regulation.txt

# 4分组
grep -Eo 'request_id=[0-9a-f-]+' regulation.txt

# 5 取消特殊含义（转义）
# .的含义是匹配人资字符串。192.168.1.10中有很多的点。我们要取消点的含义
# [0-9]{1,3}\. -> 有1个或3个数字，后面有点号
# ()\.{3} -> 有这样组合的重复三组
grep -Eo '([0-9]{1,3}\.){3}[0-9]{1,3}' regulation.txt
```

2. 单字符匹配

```
# 1. 点号 匹配任意单字符
# 输出带有id=001 / id=002 / id=003的行
grep -E 'id=00.' regulation.txt

# 2. [] 匹配集和中任意一个字符
# 匹配首字母是I或W或E; 后面跟 3~4 个大写字母
grep -Eo ' [IWE][A-Z]{3,4} ' regulation.txt


# 3. [] 范围: [a-z]、[0-9]、[A-F]、各种符号（. _ % +） 等
# 注意: [0-9]是数字 但是如果想匹配字面量- 需要将-放在最前面或者最后面
# [0-9]+ 取出连续的数字 + 前面内容重复1次或多次
grep -Eo 'cost=[0-9]+ms' regulation.txt

# 4. 取反: [^...]
# 匹配“非空格字符”连续出现（直到空格为止）
```

```
grep -Eo 'id=[^ ]+' regulation.txt
```

3. Anchoring Matches

```
# 1. ^ 行首锚定：行首必须是^后面的开头
# 1.1 只匹配标准格式日期开头的行 2026-2-9
grep -E '^[0-9]{4}-[0-9]{2}-[0-9]{2}' regulation.txt
# 1.2 匹配 2026-2-9 和 2026/02/19
grep -E '^[0-9]{4}[-/][0-9]{2}[-/][0-9]{2}' regulation.txt

# 2. $ 行尾
grep -E '0011\.2233\.4455$' regulation.txt

# 3. 整行完全匹配
grep -E '^ipv4: .+$' regulation.txt

```

4. Repetition

```
# 1. +一次或多次
grep -Eo '[0-9]+' regulation.txt
grep -Eo 'status=[0-9]+' regulation.txt

# 2. ?0次或1次
grep -E 'dates?:' regulation.txt
grep -Eo 'https?://[^ ]+' regulation.txt

# 3 {n} 必须是n次
grep -E 'id=[0-9]{3}\b' regulation.txt

# 4 {n,m} 必须是n~m次 {n,} n~无限次
grep -E 'id=[0-9]{3,4}\b' regulation.txt

# 5 * 0或多次
grep -E 'multiple *spaces *here' sample.log
```

grep 选择匹配某个模式的行

grep 会把**匹配指定正则表达式**的行复制到 **stdout**（标准输出）

```
-E: 使用扩展正则表达式语法*
-i: 匹配时忽略大小写
```

```
-v: 只显示不匹配该模式的行*
-c: 输出匹配行的数量计数
-w: 仅当模式构成一个完整单词时才匹配
-x: 仅当模式匹配整行时才匹配:
-o: 只现实能匹配到的单词*
-q: 有匹配的输出1 没有匹配输出0
```

wc 字数统计器

```
wc regulation.txt
# 输出 行数 单词数 字符数
# 输出 49 150 1917 regulation.txt
-c: 输出字符数（characters）
-w: 输出单词数（words; 由空白分隔的词/非空白串）
-l: 输出行数（lines）
```

tr 字符转换/替换

1. tr sourceChars destChars

```
# 将所有大写转换成小写 打印出来（不改源文件）
tr 'A-Z' 'a-z' < file1.txt
tr 'A-Z' 'a-z' < file1.txt > file2.txt
# 凯撒式移位: a->b, b->c, ... z->a
tr 'a-zA-Z' 'b-zaB-ZA' < text
```

2. tr 的参数

```
-c: 对 sourceChars 取补集（complement），即匹配所有不在 sourceChars 中出现的字节
-s: 压缩（squeeze）相邻重复字符（只保留第一个）
-d: 删除 sourceChars 中出现的所有字符（此时不需要 destChars）

# 删除输入中的所有数字
tr -d '0-9' < text

# 把文本拆成单词: 非字母数字视为分隔符; 每行一个单词
# -c首先将不属于字母和数字的替换成 换行符
# 可能存在多个空行，然后-s进行压缩tr -s '\n'变成一个
tr -cs 'a-zA-Z0-9' '\n' < text
```

Delimited Input 分隔符输入

> 许多过滤器（filters）都能处理按 **fields（字段）** 格式组织的文本数据——在电子表格里可以理解为**列（columns）**。

1. 用 | 分隔

```
COMP1511|2252424|Abbot, Andrew John     |3727|1|M
COMP2511|2211222|Abdurjh, Saeed         |3640|2|M
COMP1511|2250631|Accent, Aac-Ek-Murhg   |3640|1|M
COMP1521|2250127|Addison, Blair         |3971|1|F
COMP4141|2190705|Allen, David Peter     |3645|4|M
COMP4960|2190705|Allen, David Pater     |3645|4|M
```

2. 用冒号 : 分割

```
root:*:0:0:root:/root:/bin/bash
jas:*/44K0:100:100:John Shepherd:/home/jas:/bin/bash
cs1521:*:101:101:COMP1521:/home/cs1521:/bin/bash
cs2041:*:102:102:COMP2041:/home/cs2041:/bin/bash
cs3311:*:103:103:COMP3311:/home/cs3311:/bin/bash
```

cut 纵向切片

> table1.txt 中是 | 的分割
>
> table2.txt 中是 tab 的分隔
>
> cut 命令默认是以tab为分隔符

```
# 输出第一列
cut -f1 table2.txt

# 输出前三列
cut -f1-3 table2.txt

# 输出第1列和第4列
cut -f1,4 table2.txt

# 输出第3列之后的所有列（从第4列开始到末尾）
cut -f2- table2.txt

# 如果列是用 '|' 分隔的，输出前三列
cut -d'|' -f1-3 table1.txt

```

```
# （按列）输出每行前5个字符
cut -c1-5 table1.txt

```

sort 排序

```
-r: 按降序排序（reverse）
-n: 按数值排序（而不是按字典序 整数和小数都可以）
-d: 按“字典顺序”(dictionary order): 忽略非字母与非数字字符
-t c: 使用字符 c 作为列分隔符（默认: 按空白分隔/空白到非空白的转换）
-k n: 按第 n 列排序
-u 排序去重

# 优先按照k1排序 再按照K2排序
sort -k1,1 -k2,2 file

# 从第一列到第二列作为排序依据
sort -k1,2

# 优先第一列的数值部分升序排序 再第二列降序排序
sort -k1,1n -k2,2r file

# 列内字符排序
# k1.2 的意思是 从第二个字符开始到后面  A12 → key 是 12
A12 xxx
A03 yyy
B01 zzz
A20 ppp
sort -k1.2,1.3

# 排序小数部分
# 小数点算一位，并且从小数点后一位开始
011.222
301.222
sort -k1.4,k1,5
```

uniq 删除或统计重复项

并不会直接修改源文件

```
-c: 同时打印每一行重复出现的次数
-d: 只打印（每组只打印一份）重复的行
-u: 只打印只出现一次的唯一行（仅一次）

```

```
# 提取第一列，排序，统计
cut -f1 sort.txt | sort | uniq -c

# 提取第一列，去除空格行，排序，统计
cut -f1 sort.txt | grep -E '[^ ]+' | sort | uniq -c
```

sed 流编辑器

交互编辑器用于修改文件。并不会直接修改源文件

```
#
-n: 默认不输出任何行
-E: 使用扩展正则表达式
# 编辑命令
p: 打印当前行
d: 删除当前行（不打印）
s/regex/replace/: 把匹配 regex 的第一次出现替换为 replace
s/regex/replace/g: 把匹配 regex 的所有出现都替换为 replace
q: 立即终止 sed 执行（quit）

# 打印所有行
sed -n 'p' < regulation.txt
# 读到 1~10 行（到第10行退出）就立刻退出
sed '10q' < regulation.txt
# 打印前 1~10 行
sed -n '1,10p' < regulation.txt
# 打印第 81 到 100 行
sed -n '81,100p' < regulation.txt

# 正则表达式（错的）
sed -En 'status=[0-9]{3}' < regulation.txt
# 正则表达式（对的）
sed -En '/status=[0-9]{3}/p' < regulation.txt
```

注意：不能用sed同时读写文件，例如：

```
sed 's/[aeiou]//g' story.txt > story.txt   # 危险: story.txt 会被破坏
# 会清空目标文件
#正确方式: 把匹配到的所有 [] 进行替换（//是空的意思 即删除）
sed -i 's/[aeiou]//g' story.txt
sed -i 's/[aeiou]/HELLO/g' story.txt # 替换成HELLO
```

find 查找文件

join 数据库操作符

1. join的选项

```
-1 k: 第一个文件的键字段是第 k 列
-2 k: 第二个文件的键字段是第 k 列
-a N: 把文件 N（1 或 2）中无法配对的行也输出（相当于保留未匹配项）
-i: 忽略大小写
-t c: 字段分隔符是字符 c（默认通常是空白/Tab）
```

2. 指令

```
# -t' ' 用空格作为分隔行
# -2 2: 第二个文件（data2）的键在第 2 列（角色名）
# 省略了 -1 1 也就是第一个文件的键在第1列
# -a 1: 保留第一个文件中找不到匹配的行
join -t' ' -2 2 -a 1 database1.txt database2.txt
```

paste 合并文件

把多个文件按“行对齐”并排拼成一行输出（类似把多个文件当作“列”，横向合并）

```
# 合并后两列中间默认是空格
# 文件的列不一样长 则用空格补齐
paste a.txt b.txt
# -d 指定分隔符
paste -d',' a.txt b.txt
# -d 先使用, 再使用;
paste -d',:' f1 f2 f3


```

tee 管道的输出复制到一份文件中

```
# echo 输出到终端 并且通过管道输入到copy.txt文件中
echo Hello Andrew | tee copy.txt

```

xargs 用标准输入提供的参数来运行命令

总结

> **水平切片**（选取部分行）：`cat`，`head`，`tail`，`grep`，`sed`，`uniq`
>
> **垂直切片**（选取部分列）：`cut`，`sed`
>
> **替换**（Substitution）：`tr`，`sed`
>
> **聚合 / 简单统计**：`wc`，`uniq`
>
> **组装 / 合并数据源**：`paste`，`join`
>
> **重排**：`sort`
>
> **查看（通常管道末端）**：`more`，`less`
>
> **文件系统搜索**：`find`
>
> **可编程过滤器**：`sed`（以及 `awk`，`python`，`perl` 等）

Shells

Shell 核心运行机制

1. 提示符

> ```
> $
> 或
> user@host:~$
> ```

2. 读取一行输入

> ```
> ls -l /tmp
> ```

3. 做转换

> ```
> $HOME # 变量展开
> *.txt # 通配符展开
> $(date) # 命令替换
>
> echo $HOME/*.txt # 找出 home变量下面(路径) 的txt文件
>
> ```

```
先把$HOME展开成实际路径，再把*.txt展开成文件列表
```

4. 拆空白成单词

```
cp a.txt b.txt
```

5. 第一个单词为命令名，其余作为参数

```
grep hello file.txt
```

6. Shell的处理顺序

> 波浪号展开（`~z1234567` → `/home/z1234567`）
>
> ↓
>
> 参数和变量展开（`$HOME` → `/home/z1234567`）
>
> ↓
>
> 算术展开（`$((6 * 7))` → `42`）
>
> ↓
>
> 命令替换（`$(whoami)` → `z1234567`）
>
> ↓
>
> 单词拆分（按空白字符把这一行拆开）
>
> ↓
>
> 文件名展开（`*.c` → `main.c i.c`）

echo

把参数打印到标准输出(stdout)

```
-n 不输出结尾换行符
-e 启用反斜杠转义序列解释

$ echo -n "hello word"
输出:
hello word$

echo -e "hello\nworld"
```

```
输出:
hello
world
```

1. 普通输出

> echo Hello Andrew

2. 单引号阻止转义

> 输出：\n
>
> $ echo '\n'
>
> ```
> $ echo '\n'
> 输出: \n
> ```

3. `-e` 解释转义

> ```
> $ echo -e '\n'
> 输出: \n
> ```
>
> 输出：\n

4. `-n` 不加末尾换行

> ```
> $ echo -n Hello Andrew
> Hello Andrew$
> ```

Shell 变量

shell变量是无类型的→都当成字符串

shell变量不需要声明，不需要初始化

1. shell本质是字符串

> ```
> x=123
> 当成123看到。只有在算术上下文例如:
> $((...)) 才会被当成数字解释
> ```

2. 赋值语法非常严格

```
正确
name=Andrew
错误
name = Andrew
```

3. `$name` 是“取值”，`name=value` 是“赋值”

> `name=Andrew`：设置变量
>
> `echo "$name"`：读取变量内容

4. 子shell作用域

```
x=1 # 先把x设成1
echo hi | read x # 管道执行是进入子进程 不改变当前x值
echo $x
```

命令替换（命令展开）

1. 命令替换 = “把命令输出当字符串塞回来”

```
x=$(echo hi) # 先执行echo hi 将指令结果放入x中
echo "$x" # 必须要有引号要不然容易报错
```

2. `"$(...)"` 的形式

```
now=$(date)
echo $now # 没有双引号可能会报错

name="Andrew Ng"
echo "$name" # 输出Andrew Ng
echo $name # Andrew 和 Ng可能当成两个参数 所以会报错
```

单引号的用法

单引号会把其中字符整体组成一个单词。单引号内shell会当做字面值处理。

```
name=Andrew
echo '$name'
输出: $name
```

双引号的用法

- 双引号 `"` 会把其中字符整体组成一个单词
- 反斜杠可以用来转义 `$`, `` ` ``, `"`

```
$ answer=42
$ echo "The answer is $answer."
The answer is 42.

$ echo 'The answer is $answer.'
The answer is $answer.
```

<< Here Document (多行输入文档)

```
# tr指令 对 标准输入里的字符做处理
# $ tr a-z A-Z 这个操作是将小写转换成大写
# $ tr a-z A-Z <<END-MARKER 遇到END-MARKER这个停止输入
$ name=Andrew
$ tr a-z A-Z <<END-MARKER
Hello $name
How are you
Good bye
END-MARKER

输出:
HELLO ANDREW
HOW ARE YOU
GOOD BYE
```

Arithmetic 算数展开

1. `$((expression))` 会把 `expression` 作为算术表达式求值

```
$ x=8
$ answer=$((x*x - 3*x + 2))
$ echo $answer
```

```
42
```

2. 表达式中的变量可以不加 `$`

```
$((x+1))
$(($x+1))
```

3. 适合整数运算

word splitting 单词拆分

这个要注意，如果用错单双引号就会有不同效果。

有一个./print_argv.py文件：

```
# inspect how shell splits lines into program arguments (argv)
import sys
print(f'sys.argv = {sys.argv}')
```

执行shell指令：

```
w="xx   yyy     zzzz"
./print_argv.py $w
# 会输出三个参数
./print_argv.py "$w"
# 会输出一个参数
```

总结：我们用变量一般是当做一个整体来使用所以，默认应该写成 `"$var"`

`* ? [ ] !` - pathname globbing 路径名通配

1. `? [ ] !` 这些字符会让一个单词与路径名（文件名）进行匹配看起来有点像正则表达式，但功能弱得多
2. `*` 匹配任意长度（0 个或多个）的任意字符（类似正则 `.*`）
3. `?` 匹配任意一个字符（类似正则 `.`）
4. `[characters]` 匹配 `characters` 中的任意一个字符（类似正则字符类 `[]`）

5. `[!characters]` 匹配**不在** `characters` 中的任意一个字符
   （类似正则 `[^...]`）

案例：

```
a.txt  b.txt  c.log  cat.txt  data1.csv  data2.csv  img.png
echo *.txt # 找到所有的txt文件
echo c* # 找到c开头的文件
echo ?.txt # 匹配到1个文件名的文件
```

I/O Redirection 输入/输出重定向

一个命令的 **stdin（标准输入）**、**stdout（标准输出）** 和 **stderr（标准错误）** 可以被重定向到/来自文件。

1. < infile：stdin连接到文件infile
2. > outfile：把stdout写入文件outfile
3. >> outfile srdout追加到文件outfile中
4. 2> 把stderr(报错信息) 写入到outfile
5. 2>>stderr 追加到文件outfile
6. > outfile 2>$1 把 `stdout + stderr` 都发送到 `outfile`

PIpelines

就是把前一个命令的输出，当作后一个命令的输入。

```
cat file.txt | grep hello | sort
```

- 注意 管道调用子程序→管道中修改变量可能导致失败

PATH和.

PATH是环境变量，是一串目录。当输入一个命令的时候，shell会按照顺序去目录里面找

.表示当前目录。

```
# PATH=.:/bin:/usr/bin
当输入cat的时候：
1. 找./cat
```

```
2. /bin/cat
3. 再找/user/bin/cat
```

- 如果在当前目录下面放了一个cat文件
- PATH=.:/bin:/usr/bin
- 当输入cat命令的时候，会优先用当前目录的文件

总结：尽量不要把 .放到PATH路径中

Shell 脚本控制流

1 Shell Scripts

在文件中执行一个shell命令

```
$ cat hello
echo Hello, John Connor - the time is $(date)
$ dash hello
Hello, John Connor - the time is Fri 29 Aug 1997 02:14:00 EST
```

释伴行 shebang `#!`

因为加了一行#!/usr/bin/env dash，所以给脚本执行权限。就可以直接运行 ./hello 不用写 dash hello

```
$ cat hello
#!/usr/bin/env dash
echo Hello, John Connor - the time is $(date)
$ chmod 755 hello
$ ./hello
Hello, John Connor - the time is Fri 29 Aug 1997 02:14:00 EST
```

释伴行 shebang `#!`

```
# 直接制定bash的路径
写法A #!/bin/bash

# env去当前环境中的bash
```

```
写法 B（更通用）#!/usr/bin/env bash
```

2 Shell 内建变量

```
$0 命令的名字
$1 第1个命令行参数
$2 第2个命令行参数
$# 命令行参数的个数
"$@" 所有命令行参数
$? 一条命令的退出状态码
$$ 当前shell进程的进程ID
```

3 词频统计

把一串Unix命令连成一个文本处理流水线。

```
#!/bin/dash
# 统计每个不同单词出现的次数
# 在给定的文件中统计（如果没有参数，则从标准输入读取）
# 例如: word_frequency.sh dracula.txt
# 由 andrewt@unsw.edu.au 编写，作为 COMP(2041|9044) 示例
cat "$@" |                      # tr 不接受文件名作为参数
tr 'A-Z' 'a-z' | '[:upper:]' '[:lower:]'    # 将大写映射为小写，优于使
tr ' ' '\n' |                   # 转换为每行一个单词
tr -cd "a-z'\n" |               # 删除除 a-z 和 ' 之外的所有字符
grep -E -v "^\s*$" |            # 删除空行
sort |                          # 按字母顺序排列单词
uniq -c |                       # 统计每个单词出现次数
sort -rn                        # 按出现频率降序排列
# 注释:
# - 前两个 tr 命令可以合并
# - 可以用 sed 's/ /\n/g' 替代 tr ' ' '\n'
# - 可以用 sed 's/[^a-z']//g' 替代 tr -cd "a-z"
```

4 调试Shell脚本

1. 在命令行中先测试脚本的部分内容
2. 使用 `echo` 打印变量的值
3. 添加 `set -x` 来查看正在执行的命令
4. 退出状态与控制：0是正常 1不正常

5. test 命令：成功返回0 不成功返回非0

```
# 变量 msg 的值是否为 "Hello"?
test "$msg" = "Hello"

# x 是否包含一个大于 y 的数值?
test "$x" -gt "$y"

# 错误示例: 会展开成 "test hello there = Hello"?
msg="hello there"
test $msg = Hello

# x 的值是否在 10..20 范围内?
test "$x" -ge 10 -a "$x" -le 20

# 文件 xyz 是否是一个可读目录?
test -r xyz -a -d xyz

# 另一种语法; 需要右方括号 ]
[ -r xyz -a -d xyz ]
```

4 if 语法

if 是 判断这个语句是否成功执行的。

: 是空命令用来占位

```
if command1
then
    then-commands
elif command2
then
    elif-commands
else
    else-commands
fi
```

先执行comand1

- 如果退出状态是0，则执行then-commands。
- 如果退出状态是非0，则执行command2
  - 如果command2 执行状态是0 则执行 elif-commands

5 while语法

```
while command
do
    body-commands
done


#!/bin/dash
# /usr/bin/seq 的一个简单模拟，用作 COMP(2041|9044) 示例
# andrewt@unsw.edu.au
# 打印 1..n（不做参数检查）
# ./script.sh 3 last=第一个参数
last=$1
number=1
while test $number -le "$last"
do
    echo $number
    number=$((number + 1))
done
```

6 使用退出状态进行条件执行

1. 命令之间用；分割

   > 则命令都会执行

2. 命令之间用 && 分割

   > 则只要某个命令非0，执行就会停止

3. 命令之间用 || 分隔符

   > 只要有个命令返回为0(执行成功)就停止

4. { }将命令分组

5. () 将命令分组 但在子shell中执行。

```
# 如果 a.out 存在且可执行，就运行它
test -x a.out && ./a.out

# 如果目录 tmp 不存在，就创建它
test -d tmp || mkdir tmp

```

```
# 如果目录 tmp 不存在，就创建它
{test -d tmp || mkdir tmp;} && chmod 755 tmp

# 但更简单的做法是
mkdir -p tmp && chmod 755 tmp
```

Week-lab心得

掌握指令的用法：

```
# 1. 正则化语法
# 1.1 参数用法
-v 取反
-o 找到严格匹配的 而不是一行
# 1.2 正则化语法
(TODO|FIXME) 两个词二选一
[[:space:]]  代表空格或是tab

# 2. sort语法
# 表格用 | 分开
sort -t '|' file
# 先排序第一列 再排序第二列
sort -t '|' -k1,1 -k2,2 file
# 从第一列到第二列作为排序依据
sort -t '|' -k1,2 file
# 第一列递减排序 并且 第二列按照数值排序
sort -t '|' -k1,1r -k2,2n file
# 列内按照字符排序
# 例如一条数据： 201.333 要求排序小数部分
sort -t '|' -k1.4,k1.7 file
# 排序去重
# 与uniq的区别。uniq会删除所有的不重复的，保留重复的并计数。
# sort -u 是去除重复的。
sort -uk1,1

# 3. wc 计数
wc -l # 统计行 经常使用 换句话说就是统计数据的个数

# 4. cut 提取列数据
cut -d '分隔符' -f 列号 文件名
echo '123 abc:def:ghi' | cut -d':' -f1 # 输出123 abc
echo '123 abc:def:ghi' | cut -d':' -f2 # 输出123 def
echo '123 abc:def:ghi' | cut -d':' -f3 # 输出123 ghi

# 5. 统计一列中同类的数量
```

```
cut -f1 file | sort | uniq -c
cut -f1 file | sort | uniq -c | sort -nr
uniq -d # 只去重不统计行号

# 6. head 和 tail
head -n 3 # 显示前三条

# 7. sed流编辑器的使用
# 7.1 常用替换命令s(全局替换) g是全局的意思
sed 's/旧内容/新内容/g' 文件
sed -E 's/旧内容/新内容/g' 文件 # 正则化匹配
# 7.2 删除内容
# 方式1 用d 删除（优点可以删除整行 包括换行符）
sed '条件d' 文件名
sed '3d' file.txt # 删除第三行
sed '1,5d' file.txt # 删除1-5行
sed '$d' file.txt # 删除最后一行
# 方式2（推荐）
sed 's/正则表达式//g'
sed -E '/正则/d' file
# 7.3 sed 范围删除
sed '/起始模式/,/结束模式/d' file.c
sed '/int main/,/\}/d' file.c

# 7.4 查找内容 -n '/正则表达式/p'
sed -n '/正则表达式/p' 文件
sed -En '/extern/p' program.c

```

W3

lab03 心得

1 易错点

1. 条件：[]中两边必须有空格
2. 条件：[]中的变量必须是 双引号""
3. 条件：空语句是用 ： 冒号替代
4. 赋值：= 左右两变不能有空格
5. 赋值：被赋值的变量不能夹$
6. 整个代码最后要有exit0 如果是错误则exit1

2 if 语法

1. 基本语法

   - if/elif then
   - else 不用then
   - 结尾要用if

```
if [ condition1 ]
then
  command1
elif [ condition2 ] || [condition3] # 或关系 与关系是&&
then
  command2
else
  command3
fi
```

2. 条件判断

判断条件	含义
-eq	= equal
-ne	≠ not equal
-lt	< less than
-le	≤ less than or equal
-gt	> great than
-ge	≥ great than or equal

```
#!/bin/dash
# $# 就是 参数个数的意思
if [ "$#" -eq 2 ]
then
    :
else
    echo "Usage: ./echon.sh <number of lines> <string>"
    exit 1
```

```
fi

if [ ! -f "$file"] # 如果不是普通文件就继续执行
then
  command(continue)
fi

```

3 case语法

1. 结尾要用esac
2. 基本语法

```
case "$变量" in
    模式1)
        命令1
        ;;
    模式2)
        命令2
        ;;
    *)
        默认命令
        ;;
esac
```

3. 模式（shell模式，类似正则）
   - * 任意长度的任意字符
   - ? 任意一个字符

建议背

```
# 第一个参数不是数字
case "$1" in
    ''|*[!0-9]*)
        echo "./echon.sh: argument 1 must be a non-negative integer"
        exit 1
        ;;
esac
```

4 while语法

1. while 条件 do 命令
2. while语法，结尾要用done

倒数循环背下

```
n="$1"
while [ "$n" -gt 0 ]
do
    echo "$2"
    n=$((n-1))
done
```

5 for语法

1. for 变量名 in 列表
2. for的语法，结尾要用done

```
for 变量名 in 列表
do
    命令
done
```

6 命令替换

变量 = 一个命令的输出

```
变量 = $(指令)

n=$(wc -l < "$file")  # 输入重定 将file的内容输入到wc中
bytes=$(wc -c < "$file")  # 统计文件的字节数量

```

W4 复习 常用指令

1 正则化

1.1 grep的参数

- 用扩展正则表达式、只输出匹配内容、取反、静默模式(找到0 没找打1 错误2)

grep -E
grep -o

> grep -v
> grep -q

1.2 RER

- POSIX字符类：匹配空白字符、行匹配数字、匹配字母

> [[:space:]]
> [[:digit:]]
> [[:alpha:]]

2 shell指令

2.1 基本文件操作指令

- 创建文件夹、创建多层文件夹、删除空文件夹、删除文件夹以及文件夹下面的所有内容、删除文件夹下面的内容但保留文件夹、重命名文件夹

```
# mkdir 如果已经创建目录，再创建就会报错
mkdir 目录名
mkdir -p 目录1/目录2/目录3
rmdir empty_dir
rm -rf 目录名
rm -rf 目录名/*
mv 旧文件夹名 新文件夹名
```

- 创建文件、查看文件内容、修改文件内容、删除文件、重命名文件

> torch file.txt
>
> cat file.txt
>
> nano file.txt
>
> rm file.txt
>
> mv file1.txt file2.txt

- 将一个文件夹下面的txt文件**移动/粘贴**到另一个文件夹中

```
mv file1/*.txt file2/
```

```
cp file1/*.txt file2/
```

2.2 基本指令

- 统计数据数量或统计行数量、统计字节数

```
wc -l file
wc -c file
```

- 字符转换/替换

```
tr 'a-z' 'A-Z' # 小写变大写
```

- cut提取列数据：提取第1列、提取1-3列、提取1和5列、提取用分割符的第1列

```
cut -f1 txt
cut -f1-3 txt
cut -f1,5 txt
cut -d '|' -f1 txt
```

- sort排序的参数：降序排序、数值排序(按照数值大小排序)、排序后去重

```
sort -r txt
sort -n txt
sort -u txt
```

- sort排序指令：优先第一列排序再第二列排序的、把第一列和第二列当作一个排序、第一列数值排序、第一列倒序排序

sort -k1,1 -k2,2 txt

sort -k1,2

sort -k1,1n

sort -k1,1r

- sort列内字符位置：提取102.345的小数部分、提取102.345的整数部分

sort -k1.5

sort -k1.1, 1.3

- 保留重复项、统计重复项、仅去重

> sort | uniq -d
> sort | uniq -c
> sort -u

- sed流编辑器：查找、替换、删除整行、删除匹配内容、删除范围的内容

```
# -n的含义是输出先要的行
sed  -En '/正则表达式/p' 文件
sed -E 's/旧内容/新内容/g'
sed -E '/正则表达式/d'
sed '/start/,/end/d'
```

- cmp按照字节比较两个文件，cmp静默模式(if)

```
cmp a.txt b.txt

if cmp -s a.txt b.txt
then
  echo "文件相同"
else
  echo "文件不同"
fi
```

2.3 其他指令

- read从标准输入读入一行

```
read var
echo "变量 = $var"
```

- date的用法，要求输出：Wed Mar 25 14:30:12 2026

```
date '+%a %b %d %H:%M:%S %Y'

%a -> 星期
%b -> 月份缩写
%d -> 两位日期

```

```
%H -> 小时24小时
%M -> 分钟
%s -> 秒
%Y -> 年
```

3 shell脚本

- shell中特殊变量：获取脚本名称，获取第1个参数、获取第2个参数、获取参数个数、获取所有命令参数、获取上一条指令执行的状态码

```
"$0"
"$1"
"$2"
"$#"
"$@"
status=$? #必须紧贴的要测试的指令(上一条指令)
```

- 变量赋值和变量使用

```
par=20 # 等号两边不能有空格
echo "$par" # 你有有双引号+必须有$
```

3.1 条件判断（if）

> 在shell中if时根据 退出状态码判断是否执行的。如果退出状态时0就执行if中搞到内容

- 条件判断基础语法

```
if [ 条件 ] || [ 条件 ] # 必须两边有空格   而且||是有顺序的
then
  命令1
elif [ 条件 ] && [ 条件 ]
then
  命令2
else    # 只有else不需要then
  命令3
fi
```

- 条件数值大小：条件判断的格式，等于，不等于，大于，大于等于，小于，小于等于

```
if [ "$par" -eq 5]

-eq
-ne
-lt
-le
-gt
-ge
```

- 条件的文件判断：判断是否是普通文件，判断文件是否存在，判断路径是否存在

```
if [ -e "$变量" ] 文件存在
if [ -f "a.txt" ] 存在且是普通文件
if [ -d "dir1" ] 存在且是目录(目录的名字)
if [ -n "a.txt "] 变量非空
-z 变量为空
```

3.2 循环

- for遍历列表、遍历文件夹下所有文件，遍历文件夹下txt文件，遍历参数文件夹下面的所有文件，遍历所有参数

```
# for遍历列表
for 变量 in 值1 值2 值3
do
  命令
done

# 遍历文件夹下所有文件
for 变量 in *
do
  命令
done

# 遍历文件夹下txt文件
for 变量 in *.txt
do
  命令
done

# 遍历参数文件夹下面的所有文件
```

```
for 变量 in "$path"/*  # 文件夹/*
do
  命令
done

# 遍历所有参数
for 变量 in "$@"
do
  命令
done
```

- for循环的单词分割：for循环不适用用单词分割，for使用单词分割

```
for i in "$x"

# 如果变量是类似于 "1.txt 2.txt" 就需要使用单词分割
for i in $x
```

- while遍历

```
i=1
while [ "$i" -le 3]
do
  echo "$i"
  i=$((i+1))
done
```

3.3 case语法

- 基本语法

```
case 变量 in
  模式)
    命令
    ;;
  *)
    命令
    ;;
```

```
esac
```

- 判断第一个参数是不是字符

```
case "$1" in
  ''|*[0-9]*)
    命令
    ;;
esac
```

3.4 参数展开

```
file_name="abc"
# 我们希望读取一个叫abc.txt的文件
cat "${file_name}.txt"

# 拼接路径的时候也会用到
test_dir="./my/test"
test_file="teat03.txt"
test_path="${test_dir}/${test_file}"
```

3.5 用退出状态控制后续命令

```
command1 && command2  前面的成功(退出码是0)后面的才执行

command1 || command2 前面的失败(退出码是1)后面的才执行
```

4 输入/输出重定向

重定向→通过命令直接读写文件

1. 输出重定向

```
echo hello > a.txt # 将"hello"覆盖到a.txt中
echo hello >> a.txt # 将"hello"追加到a.txt中
```

```

```

2. 输入重定向

```
command < 文件(文件路径)

wc -l < ./a.txt
sort < a.txt
```

3. 错误输出，错误输出和正常输出分开保存，错误和正常保存到一个文件中

```
ls no_such_file 2> err.txt

ls no_such_file > out.txt 2> err.txt

ls no_such_file > output.txt 2>&1

echo "hello word" > out.txt 2>err.txt # 正确的
```

怎么在.sh中调用.sh文件

- 如果是自己写代码：./*.sh
- 如果是测试或提交：*.sh

5 例程

1. 找出两个文件夹中任意两个文件名和内容相同的文件

```
外层for循环遍历dir1
  [ dir2中也有dir中的文件 ] || continue
  cmp -s 比较两个文件内容
```

2. 找到文件名/文件夹名最大(数字)的文件

```
# 文件夹
```

```
max=$(ls dir | sed -E 's/\.txt$//'| sort -nr | head -n 1)
```

```
要执行的文件名 参数(数字) < 输入数据的文件路径(不能是数据必须是文件，所以这一块必须是文件路径) > 输出结果的文件路径 2>输出报错的文件路径

"$filename" $arguments < $stdin >$actual_stdout 2>"$actual_stderr"
```